



ALBUKHARY INTERNATIONAL UNIVERSITY

CCC2113 Data Structure and Algorithm Analysis

Sorting Algorithm

Prof Dr Zurinahni Zainol

Objectives

- Learn the various sorting algorithms
- Explore how to implement selection sort, Insertion sort, Quicksort, Mergesort, and Heapsort

Selection Sort: Array-Based Lists

- **Selection Sort** is a comparison-based sorting algorithm.
- It sorts an array by repeatedly selecting the **smallest (or largest)** element from the unsorted portion and swapping it with the first unsorted element.
- This process continues until the entire array is sorted.

Selection Sort: Array-Based Lists

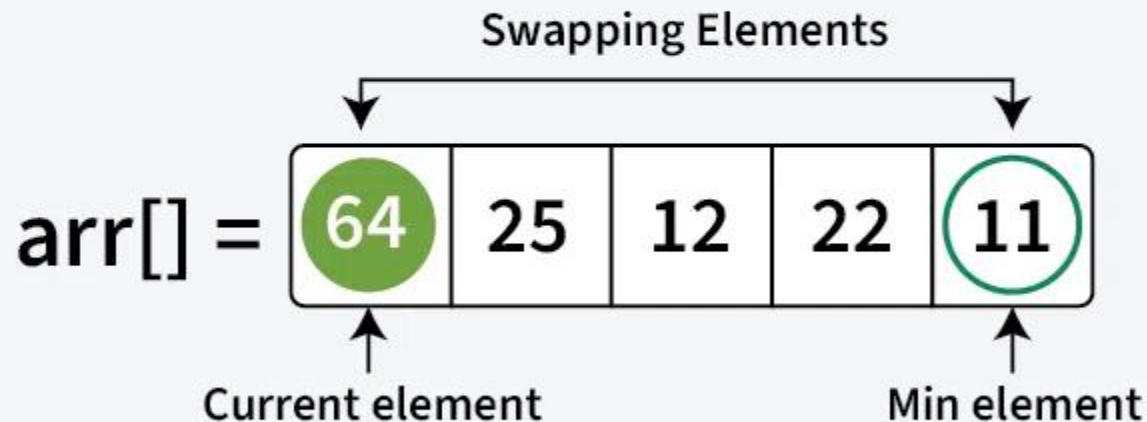
- Selection sort steps
 - In the unsorted portion of the list
 - Find location of smallest element
 - Move smallest element to beginning of the unsorted list
- Keep track of unsorted list portion with a `for` loop

```
for (index = 0; index < length - 1; index++)  
{  
    1. Find the location, smallestIndex, of the smallest element in  
       list[index]...list[length - 1].  
    2. Swap the smallest element with list[index]. That is, swap  
       list[smallestIndex] with list[index].  
}
```

Selection Sort

01 Step

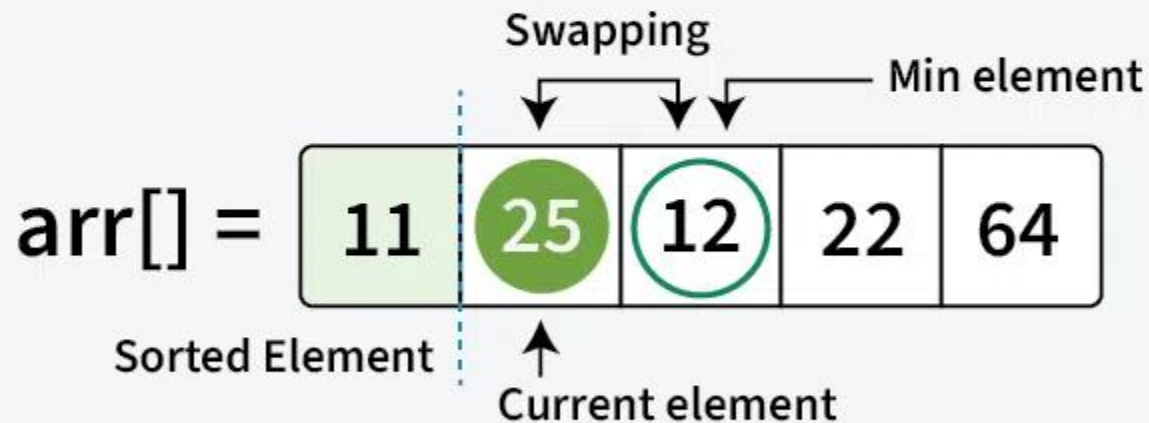
Start from the first element at index 0, find the smallest element in the rest of the array which is unsorted, and swap (11) with current element(64).



Selection Sort Algorithm

02
Step

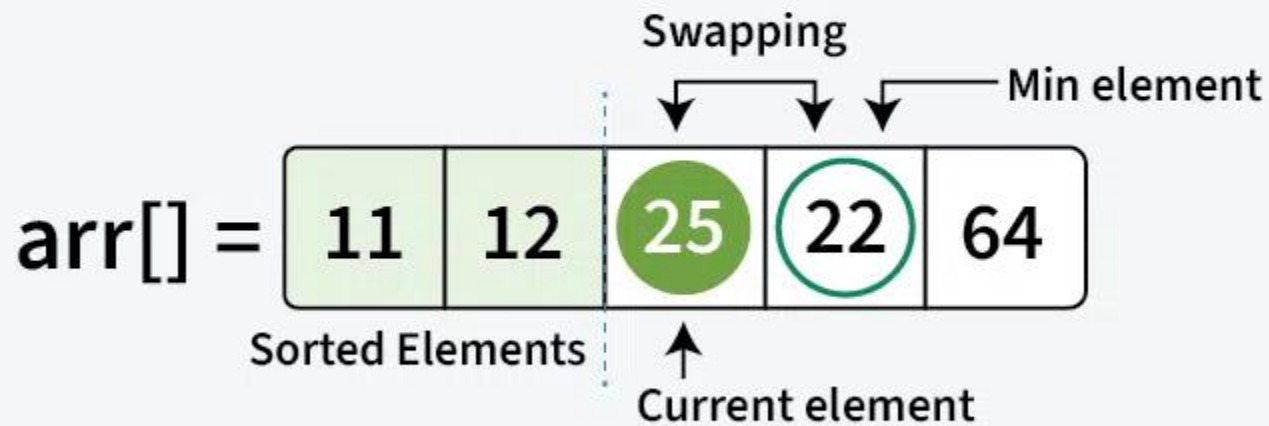
Move to the next element at index 1 (25). Find the smallest in unsorted subarray, and swap (12) with current element (25).



Selection Sort Algorithm

03
Step

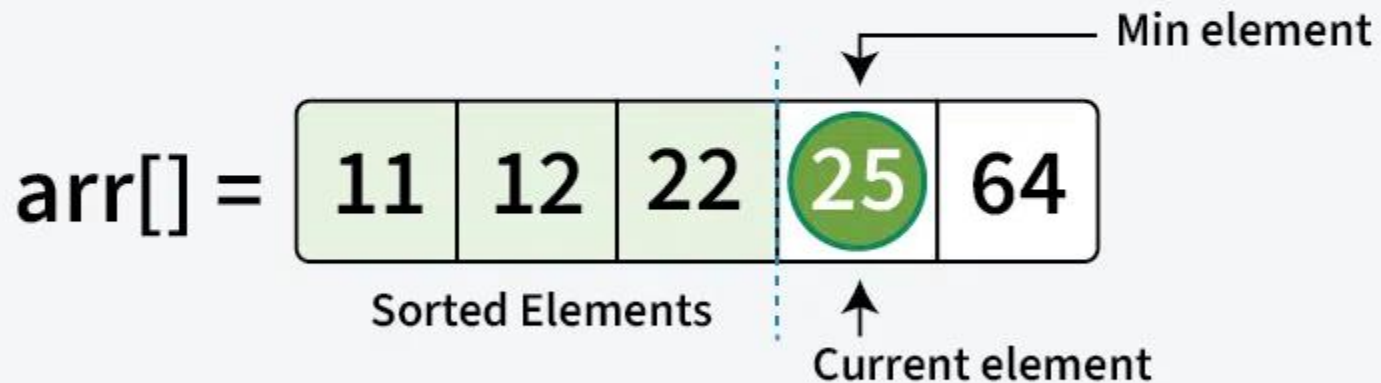
Move to element at index 2 (25). Find the minimum element from unsorted subarray, Swap (22) with current element (25).



Selection Sort Algorithm

04
Step

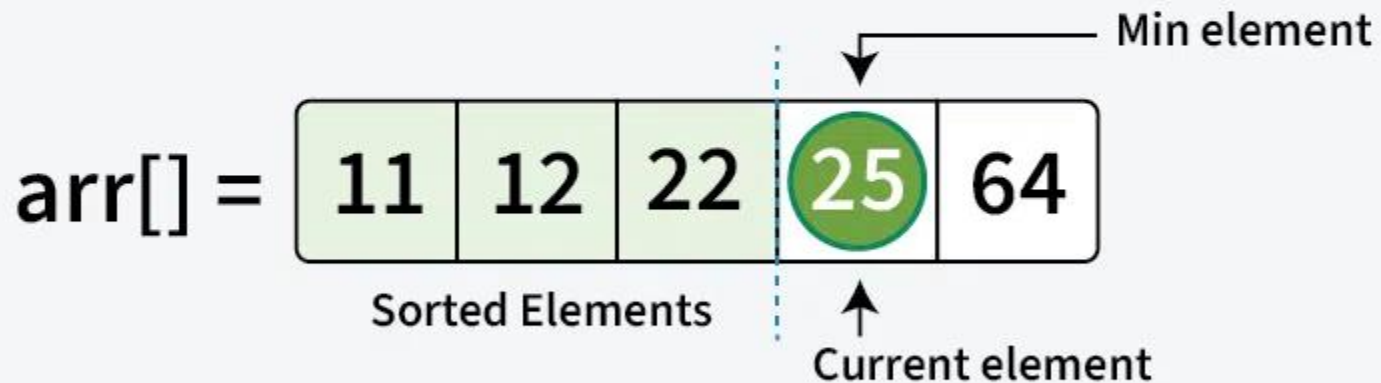
Move to element at index 3 (25), find the minimum from unsorted subarray and swap (25) with current element (25).



Selection Sort Algorithm

04
Step

Move to element at index 3 (25), find the minimum from unsorted subarray and swap (25) with current element (25).



Selection Sort Algorithm

06
Step

We get the sorted array at the end.

`arr[] =`

11	12	22	25	64
----	----	----	----	----

Sorted array

Selection Sort Algorithm

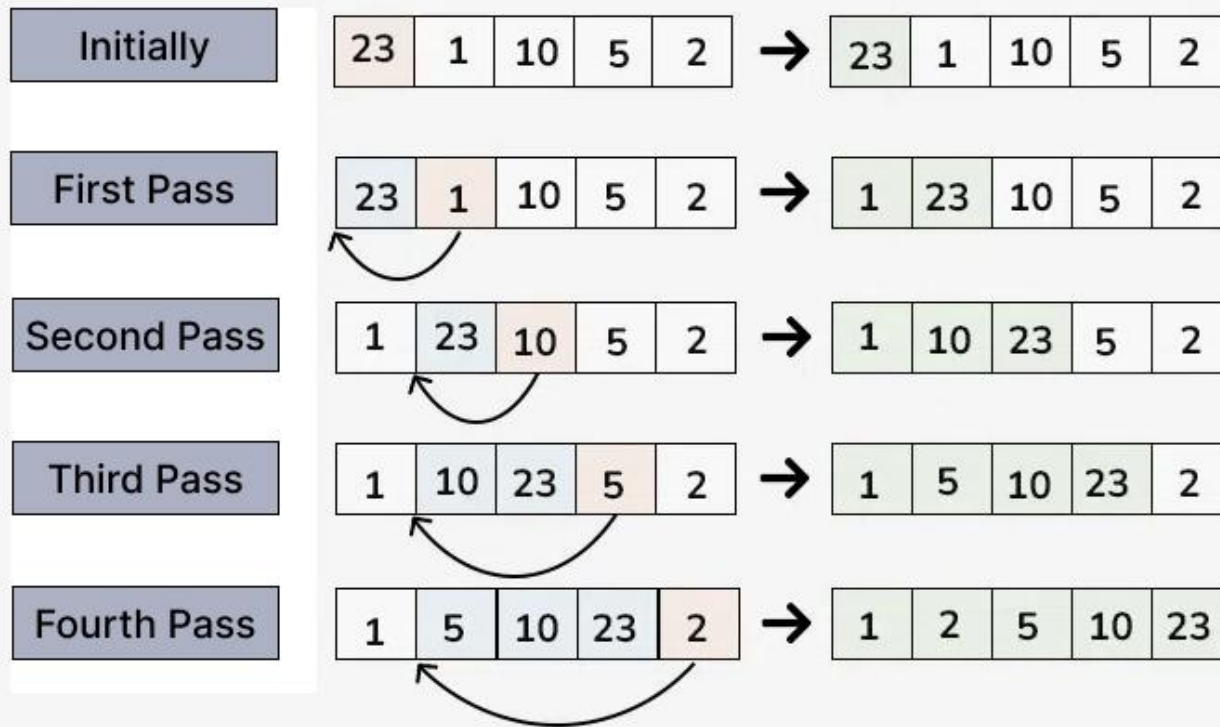
Analysis: Selection Sort

- Search algorithms
 - Concerned with number of key (item) comparisons
- Sorting algorithms
 - Concerned with number of key comparisons and number of data movements
- **Time Complexity: $O(n^2)$** ,as there are two nested loops:
- One loop to select an element of Array one by one = $O(n)$
- Another loop to compare that element with every other Array element = $O(n)$
- overall complexity = $O(n) * O(n) = O(n*n) = O(n^2)$

Insertion Sort: Array-Based Lists

- Attempts to improve high selection sort key comparisons
- Sorts list by moving each element to its proper place

Insertion Sort Algorithm



Insertion Sort



Algorithm: Insertion Sort

- Assume the **first element** of the array is already sorted.
- Start from the **second element** (index 1).
- Store the current element as **key**.
- Compare the key with elements in the sorted portion (to its left).
- **Shift all elements greater than the key one position to the right.**
- Insert the key into its **correct position**.
- Repeat until all elements are processed.

Pseudocode

```
InsertionSort(A, n)
for i = 1 to n-1
    key = A[i]
    j = i - 1
    while j >= 0 and A[j] > key
        A[j + 1] = A[j]
        j = j - 1
    A[j + 1] = key
```

Example

Input Array: [8, 3, 5, 2]

Pass-by-Pass Execution

Pass 1 (i = 1, key = 3)

Sorted part: [8]

Compare 3 with 8 → shift 8

Insert 3

Result: [3, 8, 5, 2]

Pass 2 (i = 2, key = 5)

Sorted part: [3, 8]

Compare 5 with 8 → shift 8

Insert 5 after 3

Result:[3, 5, 8, 2]

Pass 3 (i = 3, key = 2)

Sorted part: [3, 5, 8]

Shift 8, 5, 3

Insert 2 at the beginning

Result: [2, 3, 5, 8]

Final Sorted Array

[2, 3, 5, 8]

Complexity Analysis: Insertion Sort

Time Complexity of Insertion Sort

- **Best case: $O(n)$** , If the list is already sorted, where n is the number of elements in the list.
- **Average case: $O(n^2)$** , If the list is randomly ordered
- **Worst case: $O(n^2)$** , If the list is in reverse order

Advantages of Insertion Sort

- Simple and easy to implement.
- **Stable** sorting algorithm.
- Efficient for small lists and nearly sorted lists.
- Space-efficient as it is an in-place algorithm.

Disadvantages of Insertion Sort

- Inefficient for large lists.
- Not as efficient as other sorting algorithms (e.g., merge sort, quick sort) for most cases.

Quicksort: Array-Based Lists

- Uses the **divide-and-conquer technique** to sort a list
 - List partitioned into **two** sublists
 - Two sublists sorted and combined into one list
 - Combined list then sorted using quicksort (recursion)
- Trivial to combine sorted `lowerSublist` and `upperSublist`
- All sorting work done in partitioning the list

```
if (the list size is greater than 1)
{
    a. Partition the list into two sublists, say lowerSublist and upperSublist.
    b. Quicksort lowerSublist.
    c. Quicksort upperSublist.
    d. Combine the sorted lowerSublist and sorted upperSublist.
}
```

Quicksort: Array-Based Lists

- Pivot divides list into two sublists
 - lowerSublist: elements smaller than pivot
 - upperSublist: elements greater than pivot
- Choosing the pivot
 - lowerSublist and upperSublist nearly equal

	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]
list	45	82	25	94	50	60	78	32	92

List before the partition

	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]
list	32	25	45	50	82	60	78	94	92
	← lowerSublist →				← upperSublist →				

After the partition

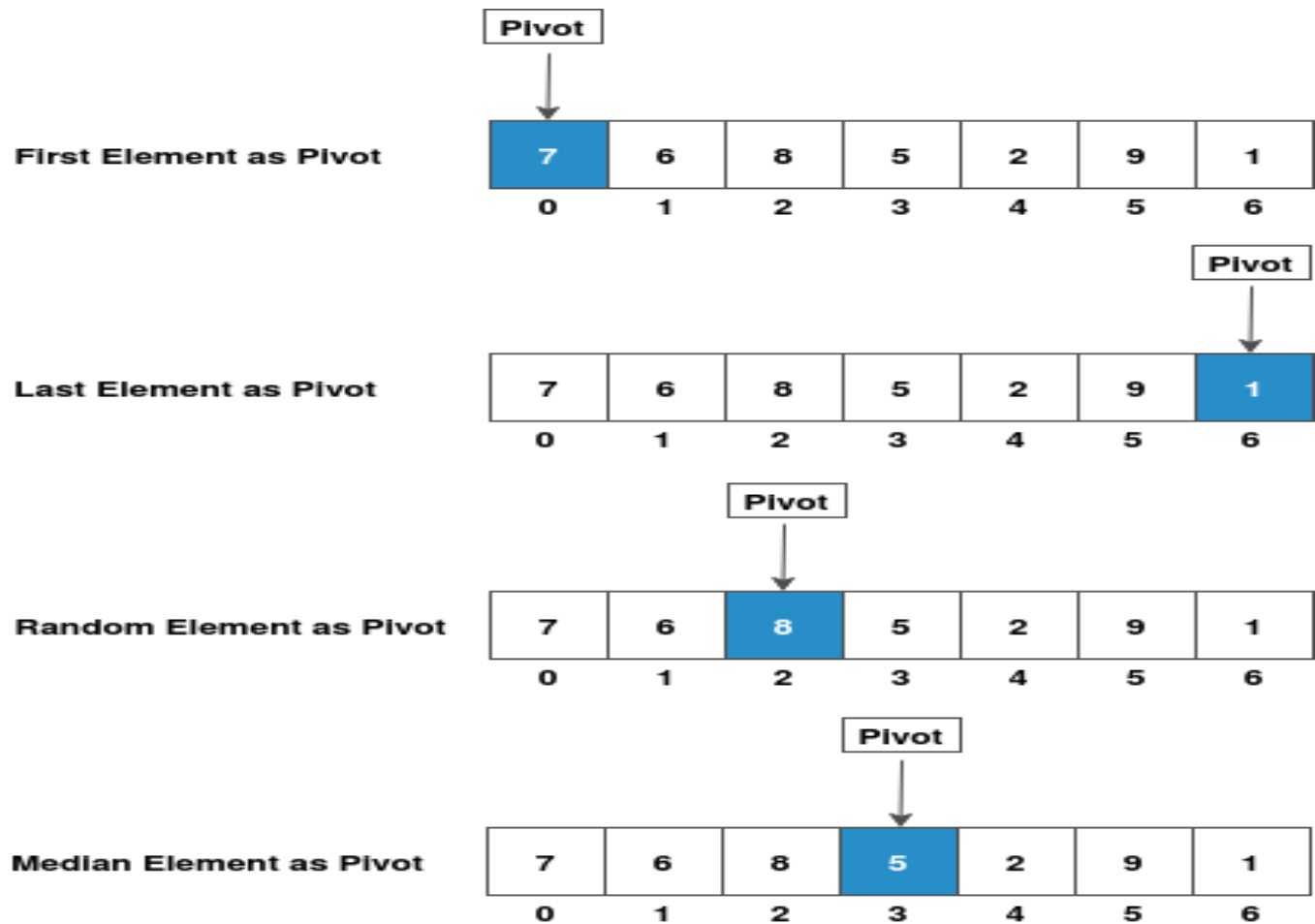
Quicksort: Array-Based Lists

- Partition algorithm
 - Determine pivot; swap pivot with first list element
 - Suppose index `smallIndex` points to last element smaller than pivot. `smallIndex` initialized to first list element
 - For the remaining list elements (starting at second element): If current element smaller than pivot
 - Increment `smallIndex`
 - Swap current element with array element pointed to by `smallIndex`
 - Swap first element (pivot) with array element pointed to by `smallIndex`

Quicksort: Array-Based Lists

- Function `partition`
 - Passes starting and ending list indices
 - Swaps certain elements of the list

How to choose PIVOT?



4	2	6	5	3	9
---	---	---	---	---	---

4	2	6	5	3	9
---	---	---	---	---	---

L

R

4	2	6	5	3	9
---	---	---	---	---	---

L **R**

4	2	6	5	3	9
---	---	---	---	---	---

L

R

4	2	6	5	3	9
---	---	---	---	---	---

L

R

4	2	3	5	6	9
---	---	---	---	---	---

L

R

4	2	3	5	6	9
---	---	---	---	---	---

↑ L ↑ R

4	2	3	5	6	9
---	---	---	---	---	---

R

L

LEFT SUBARRAY

[4 | 2 | 3]

Step 1 – Determine pivot

Step 2 – Start pointers

Step 3 – Move L (while $\text{arr}[L] < \text{pivot}$)

4 < 2 ? → **STOP** (L stays at 4)

Step 4 – Move R (while $\text{arr}[R] > \text{pivot}$)

3 > 2 move R left

2 == 2 → STOP

Step 5 – Swap values at L and R

Swap 4 ↔ 2

Step 6 – Move pointers inward

Pointers **cross** → **STOP partition**

Result after partition

[2] [4 | 3]

[4 | 2 | 3]

↑

Pivot

[4 | 2 | 3]

↑

L

↑

R

[4 | 2 | 3]

↑

L

↑

R

[2 | 4 | 3]

[2 | 4 | 3]

↑

R

↑

L

CONTINUE

Result after partition

[2] [4 | 3]

Now sort subarray [4 | 3]

Step 7 – Determine pivot

Pivot = 4

Step 8 – Start pointers

Step 9 – Compare & swap

4 > 3 → swap

Pointers cross → STOP

Final LEFT SORTED PART

[4 | 3]

↑
Pivot

[4 | 3]

↑ ↑
L R

[3 | 4]

[2 | 3 | 4]

[2 | 3 | 4 | 5 | 6 | 9]

Analysis: Quicksort

	Number of comparisons	Number of swaps
Average case	$1.39n\log_2 n + O(n) = O(n\log_2 n)$	$0.69n\log_2 n + O(n) = O(n\log_2 n)$
Worst case	$(1/2)(n^2 - n) = O(n^2)$	$(1/2)n^2 + (3/2)n - 2 = O(n^2)$

Analysis of quicksort for a list of length n

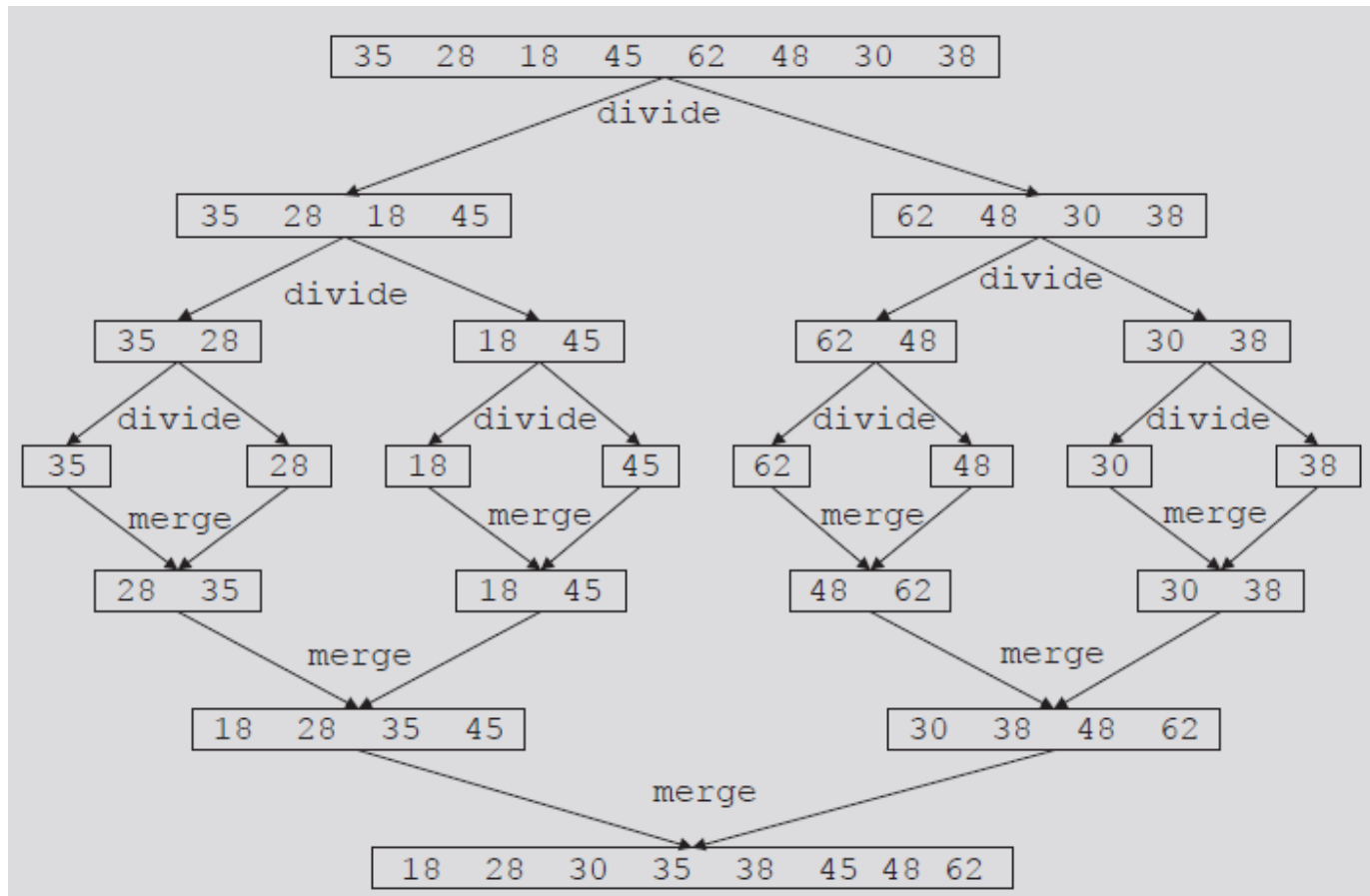
Each partition places the pivot in its correct position

Left and right subarrays are sorted recursively

Mergesort: Linked List-Based Lists

- Quicksort
 - Average-case behavior: $O(n\log_2 n)$
 - Worst-case behavior: $O(n^2)$
- Mergesort behavior: always $O(n\log_2 n)$
 - Uses divide-and-conquer technique to sort a list
 - Partitions list into two sublists
 - Sorts sublists
 - Combines sorted sublists into one sorted list
- Difference between mergesort and quicksort
 - How list is partitioned

Mergesort: Linked List-Based Lists (cont'd.)



Mergesort algorithm

Mergesort: Linked List-Based Lists (cont'd.)

- Most sorting work done in **merging** sorted sublists
- General algorithm for mergesort

```
if the list is of a size greater than 1
{
    1. Divide the list into two sublists.
    2. Mergesort the first sublist.
    3. Mergesort the second sublist.
    4. Merge the first sublist and the second sublist.
}
```

Divide

- To divide list into two sublists
 - Need to find middle node
 - Use two pointers: `middle` and `current`
 - Advance `middle` by one node, advance `current` by one node
 - `current` becomes `NULL`; `middle` points to last node
 - Divide list into two sublists
 - Using the link of `middle`: assign pointer to node following `middle`
 - Set link of `middle` to `NULL`



FIGURE 1 Unsorted linked list

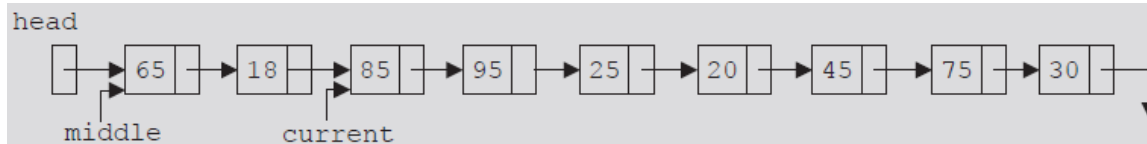


FIGURE 2 middle and current before traversing the list

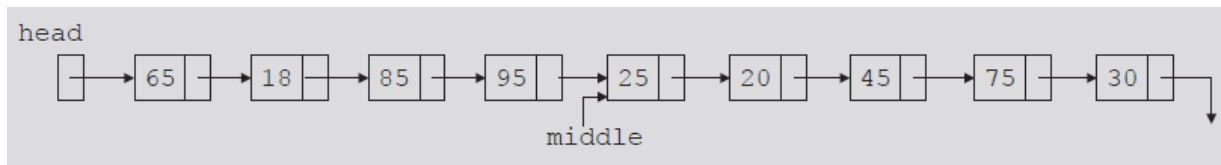


FIGURE 3 middle after traversing the list

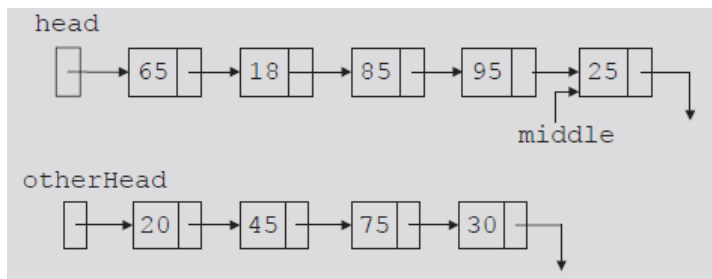


FIGURE 4 List after dividing it into two lists

Merge

- Once sublists sorted
 - Next step: merge the sorted sublists
- Merge process
 - Compare elements of the sublists
 - Adjust references of nodes with smaller info

Merge Sort follows the **Divide and Conquer** paradigm, consisting of two main phases:

35 → 28 → 18 → 45 → 62 → 48 → 30 → 38

- **Divide Phase (Top → Down)**
- The list is **recursively divided into two halves** until each sublist contains **only one element**.
- **First division**
[35 28 18 45] [62 48 30 38]
- **Second division**
[35 28] [18 45] [62 48] [30 38]
- **Third division (base case)**
- [35] [28] [18] [45] [62] [48] [30] [38]

MERGE SORT

- Division is done using **slow & fast pointers**, not array indices.
- **Merge Phase (Bottom → Up)**
- Single-node lists are **merged in sorted order**.
- **Merge step 1 (size = 1 → 2)**
- $[35] + [28] \rightarrow [28\ 35]$
- $[18] + [45] \rightarrow [18\ 45]$
- $[62] + [48] \rightarrow [48\ 62]$
- $[30] + [38] \rightarrow [30\ 38]$

- Merge step 2 (size = 2 $\rightarrow$ 4)
- [28 35] + [18 45] $\rightarrow$ [18 28 35 45]
- [48 62] + [30 38] $\rightarrow$ [30 38 48 62]
- Final merge (size = 4 $\rightarrow$ 8)
- [18 28 35 45] + [30 38 48 62]
 $\rightarrow$ [18 28 30 35 38 45 48 62]
- 18 $\rightarrow$ 28 $\rightarrow$ 30 $\rightarrow$ 35 $\rightarrow$ 38 $\rightarrow$ 45 $\rightarrow$ 48 $\rightarrow$ 62

Merge Sort performs when merging
[18 28 35 45] + [30 38 48 62]

Step-by-step merge (with comparisons)

Left (L) = 18 → 28 → 35 → 45

Right (R) = 30 → 38 → 48 → 62

Output = ∅

1) **Compare 18 vs 30**

Output: 18

2) **Compare 28 vs 30**

Output: 18 28

3) **Compare 35 vs 30**

Output: 18 28 30

4) **Compare 35 vs 38**

Output: 18 28 30 35

18 < 30 → take **18**

L → 28, R → 30

28 < 30 → take **28**

L → 35, R → 30

30 < 35 → take **30**

L → 35, R → 38

35 < 38 → take **35**

L → 45, R → 38

Merge Sort

5) Compare **45** vs **38**

$38 < 45 \rightarrow$ take **38**

- Output: 18 28 30 35 38

$L \rightarrow 45, R \rightarrow 48$

6) Compare **45** vs **48**

$45 < 48 \rightarrow$ take **45**

- Output: 18 28 30 35 38 45

$L \rightarrow \text{null}, R \rightarrow 48$

7) Left list exhausted $\rightarrow$ append remainder of Right

Append: 48 62

8) Final merged list

[18 28 30 35 38 45 48 62]

Key points

- **Comparisons happen only during merge**, not during divide
- Each step compares **current heads** of the two lists
- Smaller value is appended, pointer moves forward
- When one list finishes, **remaining elements are appended without comparison**

Why Merge Sort is Ideal for Linke Lists

Splitting	Efficient using slow/fast pointers
Merging	No shifting required, just pointer relinking
Stability	Stable sort

Analysis: Mergesort

- Maximum number of comparisons made by mergesort: $O(n \log_2 n)$
- If $W(n)$ denotes number of key comparisons
 - Worst case to sort L : $W(n) = O(n \log_2 n)$
- Let $A(n)$ denote number of key comparisons in the average case
 - $O(n \log_2 n)$

Heap Sort

Tree + Array Representation (Max Heap)

Heap Sort

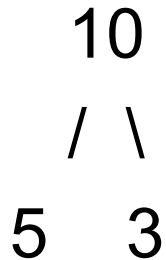
- is a **comparison-based sorting algorithm** that uses a **binary heap data structure** to sort elements efficiently.
- **Heap Sort** is a sorting algorithm that first arranges the data into a **heap** and then repeatedly removes the **root element** to produce a sorted list.
- Uses a **Binary Heap** (usually a **Max Heap** for ascending order)
- The **largest element** is always at the root

- The root is swapped with the **last element**, then the heap size is reduced
- The heap property is restored using **heapify**
- **Heapify** is an operation that rearranges elements in a heap so that the heap property is maintained.
- **Heap property** is the rule that defines the relationship between a parent node and its children in a heap.

Two types of heap property

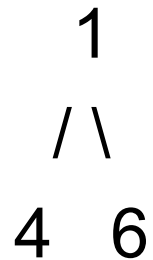
Max Heap Property

- Parent $\geq$ Left Child
- Parent $\geq$ Right Child
- The **largest element** is always at the **root**.



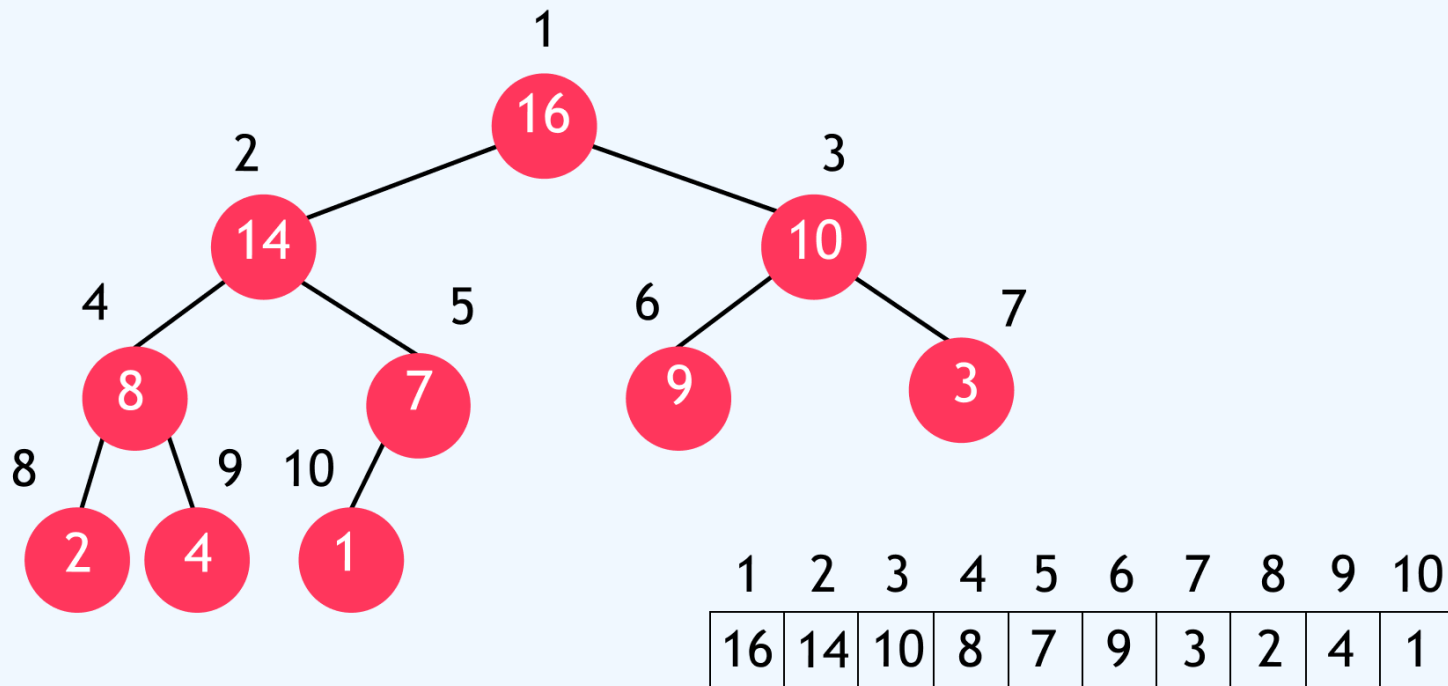
Heap Property

- Min Heap Property
- $\text{Parent} \leq \text{Left Child}$ $\text{Parent} \leq \text{Right Child}$
- The smallest element is always at the root.



HEAP SORT

What is Heap Sort in Data Structures?



Array Representation (with Indices)

how a binary heap is stored in an array, not using pointers like a linked list, but **using index calculations**.

Index: 0 1 2 3 4

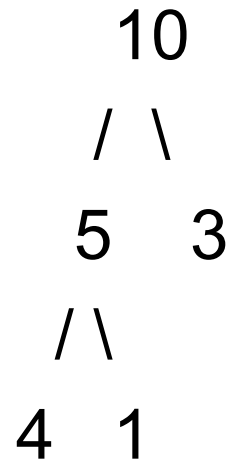
Array: [10, 5, 3, 4, 1]

Index **0** is always the **root** of the heap

This example already satisfies the **Max Heap property**

Every parent $\geq$ its children

Binary Heap as Tree



- Each node corresponds to an array index

Relationship	Formula	Meaning
Left child	$2i + 1$	Move to left subtree
Right child	$2i + 2$	Move to right subtree
Parent	$(i - 1) / 2$	Move upward

- Concrete pointer example
- node at index $i = 1$ (value = 5)

left = $2(1) + 1 = 3 \rightarrow$ value 4

right = $2(1) + 2 = 4 \rightarrow$ value 1

parent = $(1 - 1)/2 = 0 \rightarrow$ value 10

Heapify Pointer Movement

- At index i :
- Compare $\text{arr}[i]$ with $\text{arr}[\text{left}]$ and $\text{arr}[\text{right}]$
- Swap with largest child if needed
- Move pointer i downward recursively

Comparison

Algorithm	Worst Case	Extra Space
Quick Sort	$O(n^2)$	$O(\log n)$
Merge Sort	$O(n \log n)$	$O(n)$
Heap Sort	$O(n \log n)$	$O(1)$

Algorithm	Why extra space is needed
Quick Sort	Recursion stack
Merge Sort	Temporary array for merging
Heap Sort	Only index variables

Heap Sort requires $O(1)$ extra space because it performs all heap operations directly within the input array without using recursion or auxiliary storage.

Summary

- Search algorithms may require sorted data
- Several sorting algorithms available
 - Selection sort, insertion sort, quicksort, mergesort, and heapsort
 - Can be applied to either array-based lists or linked lists
- Compare algorithm performance through analysis
 - Number of key comparisons
 - Number of data movements
- Functions implementing sorting algorithms
 - Included as `public` members of the related class